

AI 调试对决实验报告

实验方式：纯人工调试 vs AI 辅助调试 对比

一、实验设计

1.1 实验目的

在系统集成阶段，从实际遇到的 Bug 中选取 3 个有代表性的案例，分别使用纯人工调试和 AI 辅助调试两种方式定位根因并生成修复方案，对比两种方式的效率、准确度和方案质量。

1.2 实验环境

- AI 工具：**DeepSeek Coder
- 项目技术栈：**Spring Boot 3 + Vue 3 + Element Plus + MyBatis + MySQL 8.0
- 调试工具：**IDEA Debugger、Chrome DevTools、Postman

1.3 实验流程

对每个 Bug，团队两人分工：

- 成员 A：**纯人工调试 — 仅靠日志、断点、代码阅读定位和修复
- 成员 B：**AI 辅助调试 — 将报错信息、相关代码片段提交给 AI，借助 AI 建议定位和修复

最后对比两组的定位耗时、准确度和方案质量。

二、Bug #1: "待标注 → 待裁定 → 可导出"状态转换有误

2.1 Bug 描述

现象：任务状态本应按"待标注 → 待裁定 → 可导出"的顺序不可逆流转。但在实际运行中，状态可以从"待裁定"直接回退到"待标注"，且"可导出"状态下的任务仍能推进到下一阶段。

环境：后端 `TaskService` 中的状态更新方法，前端 `TaskDetailView.vue` 中的阶段推进按钮

影响：核心四阶段流程的不可逆约束被打破，违反 SRS 中"流程不可逆"的 Must 级需求，数据一致性无法保证。

2.2 调试对比

步骤	纯人工调试	AI 辅助调试
1. 问题描述	在前端操作复现 Bug 后，检查后端 <code>TaskService.updateStatus()</code> 方法，发现该方法直接 <code>SET status = ?</code> 无任何前后状态校验	将 <code>TaskService</code> 中状态更新的方法代码 + 预期状态流转规则（待标注→待裁定→可导出）提交给 AI

2. 定位耗时	22 分钟 — 先怀疑前端按钮逻辑（排查 10 分钟），再排查数据库 ENUM 定义（排查 5 分钟），最后确认是 Service 层缺少状态机校验	6 分钟 — AI 指出 "方法中缺少状态机约束：① 未定义合法状态转换路径；② 未校验当前状态→目标状态是否在合法路径中；③ 建议使用状态转移表集中管理"
3. 定位准确度	准确	准确
4. 修复方案	在 <code>updateStatus()</code> 中硬编码 if-else 判断当前状态和目标状态的合法性	建议：① 定义状态转移 Map (<code>Map<String, Set<String>></code>) 集中维护合法转移；② 在更新前校验 <code>allowedTransitions.get(currentStatus).contains(newStatus)</code> ③ 补充单元测试覆盖所有合法及非法转移路径
5. 方案质量	可行但不可扩展。if-else 在状态增多时急剧膨胀，且硬编码分散在多个方法中	方案①使用状态转移表，集中、可读、可扩展；方案③建议测试覆盖边界情况
6. 最终方案	放弃原 if-else 方案，改用 AI 建议的状态转移 Map	采纳 AI 方案①②③

2.3 Bug #1 结论

- AI 对"状态机"模式有成熟认知，直接建议使用状态转移表而非 if-else，体现了良好的设计意识
- AI 的修复方案完全可用，且优于人工初版方案
- 最终方案来源：AI 建议为主

三、Bug #2：任务目录和数据目录中创建者身份显示异常

3.1 Bug 描述

现象：在任务目录页面和数据目录页面中，系统需要显示当前用户的全局角色（超级管理员/任务创建者/普通用户），但创建者（creator）身份始终无法正常显示。无论创建者角色用户登录，页面头部显示的角色身份要么为空，要么显示为"普通用户"。

环境：前端 `CreatorLayout.vue` 和 `ParticipantLayout.vue` 中的角色判断逻辑，依赖 `auth.user.role` 字段

影响：角色身份是 UI 权限判断的关键依据，显示错误导致部分功能入口（如"创建任务"按钮）显示异常。

3.2 调试对比

步骤	纯人工调试	AI 辅助调试
1. 问题描述	打开 Vue DevTools 检查 Pinia store 中 auth.user 对象，确认 role 字段值；追查登录接口返回的 LoginResponse 是否包含 role；追查后端 SysUserMapper.selectById 的 SQL 是否查询 role 字段	将前端角色判断代码 (auth.user.role)、LoginResponse DTO 定义、后端 SysUser 实体和 Mapper XML 提交给 AI
2. 定位耗时	28 分钟 — 完整链路排查：前端 store → API 响应 → Controller → Service → Mapper → SQL。最终发现在 Mapper XML 的 resultMap 中 role 字段映射到了 role_code 列，而 SysUser 实体中存储角色值的字段名为 role，字段映射名不匹配导致该字段值为 null	4 分钟 — AI 对比前端期望字段和后端返回字段后指出 "LoginResponse 中的 role 字段来自 SysUser 实体，请检查 MyBatis resultMap 是否将数据库列正确映射到实体属性名。常见原因：resultMap 中 column 属性指向了 role_code，但 property 属性写为 roleCode 而非 role"
3. 定位准确度	准确，但排查链路长	准确，直接锁定 resultMap 映射问题
4. 修复方案	修正 Mapper XML 中 resultMap 的字段映射，使 role_code 列正确映射到 role 属性	建议：① 修正 resultMap 字段映射；② 同时检查 UserVO.from() 方法是否正确转换了角色字段；③ 在前端增加兜底逻辑，若 role 为空则显示"未知角色"；④ 统一实体与 VO 间的字段名
5. 方案质量	修了根因，但未发现前端缺少兜底展示	AI 方案更全面：不仅修根因，还建议前端兜底和统一字段命名
6. 最终方案	修正 resultMap 映射，同时在前端增加兜底展示	采纳 AI 方案①②③，④待后续重构统一处理

3.3 Bug #2 结论

- 跨层排查类 Bug AI 有显著速度优势：人工需要逐层追踪（前端 → API → Controller → Service → Mapper → SQL），AI 能快速对比各层数据结构的不一致
- AI 建议的"兜底策略"（方案③）体现了防御性编程思维，人工在修复时容易只关注根因而忽略容错

- 最终方案来源：AI 建议为主

四、Bug #3：操作按钮只显示"查看详情"，其他按钮全部缺失

3.1 Bug 描述

现象：任务列表中每条任务只显示一个"查看详情"按钮，而按设计应根据任务状态和用户角色显示不同操作按钮：标注员在"待标注"阶段应看到"开始标注"，裁定者在"待裁定"阶段应看到"进入裁定"，任务创建者在"可导出"阶段应看到"查看结果/导出"。

环境：前端 `CreatorTasksView.vue` 和 `ParticipantTasksView.vue` 中的操作列模板

影响：用户无法进入标注/裁定/导出等核心操作，整个工作流无法推进。属于 P0 阻断性 Bug。

3.2 调试对比

步骤	纯人工调试	AI 辅助调试
1. 问题描述	在 Vue 模板中检查 <code>v-if / v-show</code> 的条件判断，检查 <code>taskRows.js</code> 或类似工具函数中生成按钮列表的逻辑	将任务列表的 Vue 模板代码 + 按钮显示的条件逻辑（任务状态枚举、用户角色枚举）提交给 AI
2. 定位耗时	15 分钟 — 先怀疑 <code>v-if</code> 条件写死（排查 5 分钟），再检查 <code>taskRows.js</code> 工具函数中按钮生成逻辑，最后发现条件判断中比较 <code>task.status</code> 时使用了中文字符串（如 <code>=== '待标注'</code> ），但后端某次修改中将 <code>status</code> 字段从中文 <code>ENUM</code> 改为了 <code>VARCHAR</code> 后返回的值格式不一致（前后多了空格或编码差异），导致 <code>===</code> 严格相等判断始终为 <code>false</code> ，所有条件分支失效	3 分钟 — AI 指出 "多个 <code>v-if</code> 条件同时失效通常不是因为每个条件都写错了，而是比较基准有问题。建议检查：① <code>task.status</code> 的实际值与各条件中字符串是否严格相等（注意空格、全角/半角字符）；② 是否可以直接比较状态枚举值而非字符串；③ 确认后端返回的 <code>status</code> 值在接口文档中的约定值"
3. 定位准确度	准确，但排查了多个方向才找到根因	准确，直接提出"比较基准问题"的假设
4. 修复方案	在工具函数中对 <code>task.status</code> 调用 <code>.trim()</code> 后再比较	建议：① 前端比较前 <code>.trim()</code> 处理；② 后端统一返回格式，去除多余空格；③ 使用常量定义状态值（ <code>STATUS_ANNOTATING = '待标注'</code> ）避免散落字符串；④ 改为枚举或 <code>key</code> 值比较（如 <code>'annotating'</code> ）而非中文字符串

5. 方案质量	解决了当前问题，但属于"打补丁"，未解决根本原因	方案②④从根因修复（后端统一格式），方案③提升代码可维护性
6. 最终方案	前端临时.trim() + 后端统一 trim 返回 + 使用常量替换散落字符串	综合采纳 AI 方案①②③

3.3 Bug #3 结论

- AI 对"多个相同类型的条件同时失效"的模式识别很敏锐，跳过了逐个检查条件的过程，直接分析共同根因
- **AI 倾向于建议"用枚举/key 值替代中文字符串比较"，这是更工程化的做法，但需要团队评估改动成本
- 最终方案来源：AI + 人工混合

五、综合对比分析

5.1 实验记录汇总

Bug #	Bug 描述	纯人工定位耗时	AI 辅助定位耗时	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
1	任务状态转换无约束，可逆流转	22 min	6 min	准确	完全可用	AI 为主
2	创建者角色身份显示异常	28 min	4 min	准确	完全可用	AI 为主
3	操作按钮只显示"查看详情"	15 min	3 min	准确	完全可用	AI+人工混合

5.2 AI 表现分析

AI 擅长调试的 Bug 类型

- 设计模式缺失类 Bug：Bug #1 中的状态机缺失，AI 能识别出"这里缺少一个状态机"并给出成熟的模式方案
- 跨层数据映射类 Bug：Bug #2 中的 ORM 字段映射错误，AI 能快速对比各层数据结构差异
- 条件判断失效类 Bug：Bug #3 中多个 v-if 同时失效，AI 能识别"比较基准异常"的模式
- 编码/格式不一致类 Bug：中文字符串空格、全角/半角等问题，AI 训练数据中此类案例丰富

AI 不擅长调试的 Bug 类型

- 业务逻辑深层 Bug：如标注命题与关系的数据一致性校验、裁定采纳逻辑的正确性判断，AI 因缺少法律标注领域的业务知识，定位能力明显下降

- **多模块并发 Bug**：如同一账号同时持有标注员和裁定者权限时裁定界面无法载入数据，涉及权限系统、任务分配、前端路由三者的联动，AI 难以从局部代码还原全貌

AI 修复方案的典型问题

- **方案层次不区分优先级**：AI 常同时给出"快速修复 + 根因修复 + 架构优化"三类方案，但不标注哪些是必须修的、哪些是锦上添花，需人工判断
- **过度工程化倾向**：对课程项目场景偶尔建议引入状态机框架（如 Spring State Machine）、全面重构字段命名等大动作
- **修复代码中的小瑕疵**：AI 生成的修复代码偶尔引入新的边界问题（如建议 `.trim()` 但未处理 null 情况）

5.3 人工辅助的有效策略

通过实验发现，向 AI 提供以下上下文后，调试建议质量明显提升：

1. **完整的报错信息或异常堆栈**（而非仅口头描述现象）
2. **涉及的多层代码同时提交**（前端组件 + API 响应结构 + 后端实体 + Mapper/SQL）
3. **明确说明系统的业务约束**（如"四阶段流程不可逆"），帮助 AI 理解"什么是正常行为"
4. **相关数据库 DDL**（尤其是字段类型定义、约束定义）

5.4 关键数据

- AI 定位平均耗时：**4.3 分钟**，纯人工：**21.7 分钟**，AI 提速约 **5.0 倍**
- AI 定位准确率：****100%****（3/3 均准确定位根因）
- AI 修复方案完全可用率：****约 67%****（Bug #1 和 #2 的方案可直接使用，Bug #3 的方案需人工裁剪）
- 最终方案中 AI 贡献度：约 **70%**

六、实验结论

1. **AI 在代码层面的 Bug 定位速度始终远优于人工**，尤其体现在跨层排查（前端→后端→数据库）和模式识别（状态机缺失、映射错误等）场景。人工需要逐层追踪，AI 能快速比对各层数据结构。
2. ****AI 调试最有效的场景是"代码写了但写得不完整或不对"****（如缺少状态校验、字段映射错误、条件比较失效），而非"业务规则本身没有被正确理解"。对于需要业务理解的 Bug（如同一账号多权限冲突），AI 定位效果明显下降。
3. **AI 的修复方案整体质量较高但需要人工做减法**。最常见的多余建议是引入额外的框架/库或建议大规模重构，需要在课程项目的团队能力和工期约束下做筛选。
4. **主动向 AI 提供多层代码上下文比逐步追问更高效**。一次性提交前端组件、后端 Service、Mapper XML 和数据库 DDL，AI 能在一次回答中完成跨层分析，效果远好于分步追问。